# /websearch

Show, test, or query web search engines (/websearch show|test|search|help)

## Overview

`/websearch` inspects the MasterFetch search pipeline used by `mf_search`. With no arguments it behaves like `show`, printing a table of the configured search engines and whether each is enabled and currently in use. `test` launches an asynchronous diagnostic run whose result is delivered into the message window once it finishes.

## Syntax

```
/websearch                    Show the engine status table (same as show)
/websearch show               Show the engine status table
/websearch test               Run an asynchronous engine diagnostic
/websearch search <query>     Query all engines and show ranked results per engine
/websearch help               Show the help table
```

## Options / Subcommands

| Form | Description |
|---|---|
| `/websearch` | Alias of `show` |
| `/websearch show` | Prints the `Engine / Enabled / In Use / Failed` status table |
| `/websearch test` | Runs the diagnostic off the UI thread; the result arrives later and drains into the message window |
| `/websearch search <query>` | Sends `<query>` to every configured engine via `MfSearchTool::build_orchestrator` + `SearchOrchestrator::search` and prints each merged result (`position. [engine(s)] title (relevancy)` + URL) followed by a per-engine summary table listing how many results each engine contributed; consensus hits found by several engines are credited once under the combined `engine_a,engine_b` row |
| `/websearch help` | Prints the help table |

## Engine table

The `show` table lists, per engine: whether it is enabled, whether it is currently selected for use, and whether the last diagnostic run marked it as failed.

- Keyless engines that need no API key: OpenAlex (scholarly works; set `openalex_email` in `ragent.json` or the `OPENALEX_EMAIL` environment variable to join the polite pool) and Wikipedia (encyclopedia summaries).
- API-backed engines activate only when a key is configured: `langsearch_api_key`, `tavily_api_key` (or the `TAVILY_API_KEY` environment variable), `perplexity_api_key` (or `PERPLEXITY_API_KEY`), `exa_api_key` (or the `EXA_API_KEY` environment variable), and `serper_api_key` (or the `SERPER_API_KEY` environment variable). Configure these in `ragent.json`.

## Examples

`/websearch`

Prints the engine status table.

`/websearch show`

Identical to the bare form.

`/websearch test`

Starts the diagnostic in the background; a status line confirms the run began, and the per-engine results are appended to the message window when the run finishes.

`/websearch help`

Prints the help table.

`/websearch search rust async runtime comparison`

Queries every configured engine, lists each returned result as `N. [engine] title (relevancy-score)` with the URL underneath, then prints a "Results per engine" summary (per-engine contribution counts, including consensus rows for URLs more than one engine returned) and a total/raw/duration line. Blocked engines are listed at the bottom when any occurred, in `name: reason` form (e.g. `wikipedia: rate-limited`, `tavily: Tavily API returned HTTP 432`) so quota blocks are distinguishable from transient rate-limit windows. Like `test`, the search runs off the UI thread and the report drains into the message window when it finishes.

## Output

- Bare form and `show`: the engine status table plus notes on which engines are keyless and which require API keys.
- `test`: an immediate confirmation that the diagnostic started, then the diagnostic result later (the result is queued and drained on a later poll, so it can appear after subsequent activity).
- `search`: an immediate "Starting web search" acknowledgement, then the result list, per-engine summary, totals, and any blocked-engine notes. Merge cap: 100 results with a per-engine allowance of 25; several backends hard-cap their own per-request result count (e.g. LangSearch at 10), so the visible spread can never exceed the sum of those engine caps.

## Engine-level resilience (T-016)

`mf_search` retries transient engine failures at the engine level, so every caller (this command, `mf_search`, research web-gather) inherits the same behaviour:

- transient failures (Wikipedia 429 / `rate-limited`, HTTP 5xx, transport timeouts) are retried up to **2** times with exponential backoff (1s, 2s);
- account-level quota blocks are **not** retried (Serper 403, Exa 402, Tavily 432, OpenAlex daily "Insufficient budget" 429);
- engine starts are staggered by ~120 ms inside one fan-out so keyless backends (Wikipedia, OpenAlex) are not all hit at once;
- Wikipedia summary fetches are capped at 8 concurrent requests with a 150 ms start-up stagger so a deep hit-list does not burst past Wikimedia's per-IP limiter.

## Troubleshooting

- Unknown subcommand yields a warning.

## Related

- `mf_search` - the agent tool the engines back
- `mf_fetch`, `mf_crawl`, `mf_cache_clear` - the rest of the MasterFetch tool family
- `/webapi` - control the built-in HTTP API server
- `ragent.json` - where `langsearch_api_key`, `tavily_api_key`, and `perplexity_api_key` are configured